



UNIVERSITÉ

---

# Projet de bioinformatique

## Déployer et comprendre AlphaFold2

---

*Auteur :*  
Guillaume ROSIN

*Enseignant :*  
Prénom NOM

### Résumé

Résumé à venir.

# 1 AlphaFold2 : principe général

**Le problème.** La séquence d'acides aminés d'une protéine détermine sa structure tridimensionnelle [2], mais le nombre de conformations possibles est tel qu'une exploration exhaustive est impossible. On appelle cela le « problème du repliement » [3]. Plutôt que de simuler le repliement physique, AlphaFold2 [5] *apprend*, à partir des structures connues de la PDB, la relation entre séquence et structure.

**Entrée et sortie.** L'utilisateur fournit uniquement la **séquence** de la protéine (format FASTA). AlphaFold2 renvoie les **coordonnées 3D** de tous les atomes lourds (fichier PDB ou mmCIF), accompagnées de **mesures de confiance**.

**Le MSA et la coévolution.** La première étape consiste à rechercher, dans de grandes bases de séquences, des protéines *homologues* (issues d'un ancêtre commun) et à les aligner : c'est l'**alignement multiple de séquences** (MSA). Chaque ligne est une séquence homologue et chaque colonne une position équivalente de la protéine. Le MSA apporte deux informations structurales :

- la **conservation** : une position qui ne varie jamais est probablement essentielle (cœur hydrophobe, site actif) ;
- la **coévolution** : si deux résidus sont en contact, une mutation de l'un est souvent compensée par une mutation de l'autre, sinon la protéine perd sa structure. Deux colonnes qui varient *ensemble* indiquent donc deux résidus probablement proches dans l'espace [3].

Plus le MSA est profond et diversifié, plus ces signaux sont fiables. À l'inverse, une protéine « orpheline », sans homologues, est en général mal prédite.

**Les grandes étapes.** La figure 1 résume le processus :

1. **Recherche d'homologues** dans les bases de séquences (MSA) et, en option, de structures connues similaires (*gabarits*) ;
2. **Evoformer** : un réseau de neurones fait circuler l'information entre le MSA et une « représentation de paires », qui décrit pour chaque couple de résidus  $(i, j)$  leur relation spatiale probable ;
3. **Module de structure** : il transforme la représentation de paires en coordonnées 3D (squelette, puis chaînes latérales) ;
4. **Recyclage** : la structure obtenue est réinjectée dans le réseau (3 fois par défaut) pour l'affiner.

**Pourquoi des mesures de confiance ?** Une prédiction n'est pas une mesure expérimentale : sa qualité varie d'une protéine à l'autre, et même d'une région à l'autre. AlphaFold2 a été entraîné à *estimer sa propre erreur*. Il produit un score local par résidu, le **pLDDT**, et une erreur relative entre paires de résidus, la **PAE** (section ??). Sans ces scores, l'utilisateur ne pourrait pas distinguer une région fiable d'une région inventée.

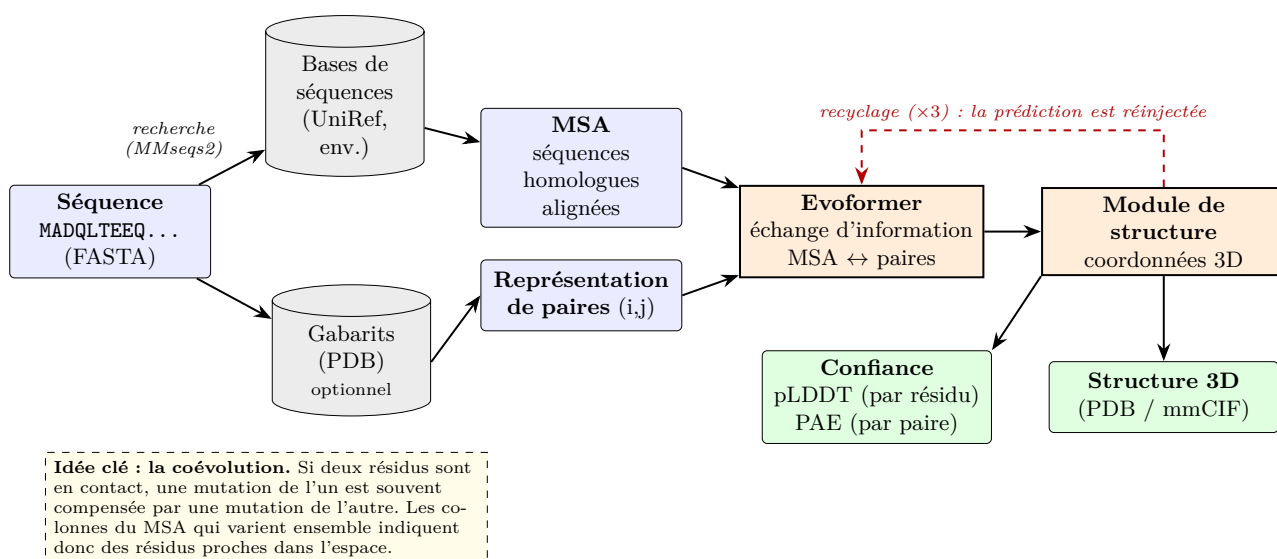


FIGURE 1 – Schéma du fonctionnement général d'AlphaFold2 (réalisation personnelle, d'après [5, 3]).

## 2 Choix technique

Le tableau 1 compare les solutions sérieusement envisagées. La machine disponible est un PC Linux (Debian 12, 12 cœurs, 14 Go de RAM, **sans GPU NVIDIA**) sur lequel Docker est installé.

|                  | Installation officielle<br>AlphaFold2 [4]                 | ColabFold sur<br>Google Colab [7]                        | ColabFold local<br>(Docker) [6]                                           | AlphaFold DB /<br>AlphaFold<br>Server [9, 1]                                   |
|------------------|-----------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| À installer      | Docker, pilotes<br>NVIDIA, scripts<br>DeepMind            | Rien (navigateur,<br>compte Google)                      | Docker + image<br>ColabFold (~10 Go) +<br>poids (~5 Go)                   | Rien (site web)                                                                |
| Calcul           | Local                                                     | Serveurs Google (GPU<br>gratuit)                         | Local (CPU ou GPU)                                                        | Serveurs EBI / Google                                                          |
| Ressources       | GPU recommandé, $\geq$<br>64 Go RAM                       | Aucune localement                                        | CPU suffisant pour <<br>300 résidus ; GPU<br>optionnel                    | Aucune                                                                         |
| Bases de données | ~2,6 To à télécharger<br>(UniRef, BFD, MGnify,<br>PDB...) | Aucune : MSA via le<br>serveur MMseqs2<br>public         | Aucune : MSA via le<br>serveur MMseqs2<br>public                          | Aucune                                                                         |
| Avantages        | Pipeline de référence,<br>hors ligne                      | Très simple, rapide<br>(GPU)                             | Reproductible (version<br>figée), scriptable, pas<br>de limite de session | Instantané                                                                     |
| Inconvénients    | Irréaliste sur notre<br>machine (disque, GPU)             | Session limitée, GPU<br>non garanti, dépend de<br>Google | Lent sur CPU ; Internet<br>requis pour le MSA                             | DB : protéines déjà<br>prédites seulement ;<br>Server = AlphaFold3,<br>pas AF2 |

TABLE 1 – Comparaison des solutions envisagées pour exécuter AlphaFold2.

**Solution retenue : ColabFold 1.5.5 exécuté localement dans Docker**, avec Google Colab documenté comme alternative dans le `README.md`. ColabFold [7] utilise *exactement les mêmes réseaux et les mêmes poids* qu’AlphaFold2. Il remplace seulement la construction du MSA (HHblits/JackHMMER sur 2,6 To de bases) par une recherche MMseqs2 [8] sur un serveur public, qui prend quelques secondes au lieu de plusieurs heures. Ce choix supprime les deux obstacles de l’installation officielle, le disque et le GPU, tout en gardant un calcul local, reproductible et automatisable. Docker fige toutes les dépendances (Python, JAX, TensorFlow) : un étudiant de l’année prochaine obtiendra le même environnement avec une seule commande. Google Colab reste la voie la plus simple pour ceux qui n’ont ni Linux ni Docker, mais nous ne l’avons pas retenu comme solution principale : les sessions sont limitées dans le temps, l’accès au GPU n’est pas garanti et le notebook évolue sans versionnage explicite.

## 3 Mise en œuvre

**Environnement.** Debian GNU/Linux 12 (noyau 6.12), processeur 12 cœurs, 14 Go de RAM, pas de GPU ; Docker 29.8.1 ; Python 3.11.2 pour l’analyse (numpy 2.4, matplotlib 3.11, dans un environnement virtuel). Aucun compte n’est nécessaire ; seule une connexion Internet est requise, pour le MSA et le téléchargement initial.

**Étapes.** Les commandes exactes figurent dans le `README.md` ; en résumé :

1. `docker pull ghcr.io/sokrypton/colabfold:1.5.5-cuda12.2.2`, pour une version figée de ColabFold ;
2. `python -m colabfold.download` dans le conteneur : téléchargement des poids AlphaFold2 (5,3 Go) dans un dossier `cache/` monté en volume, pour ne les télécharger qu’une fois ;
3. `colabfold_batch -num-models 5 séquence.fasta results/` : MSA via MMseqs2, puis prédiction par les 5 modèles AlphaFold2 (`alphafold2_ptm`) ;
4. `scripts/analyse.py` : extraction du pLDDT et de la PAE, et production des figures.

Les étapes 2 et 3 sont regroupées dans le script `scripts/predict.sh`, qui détecte automatiquement la présence d’un GPU.

### Difficultés rencontrées et solutions.

- **Pas de GPU** : l'image est compilée pour CUDA, mais JAX bascule automatiquement sur le CPU (message `no GPU detected, will be using CPU`). Le calcul fonctionne, mais il est lent : au total pour 149 résidus (voir section 4). durée
- **Fichiers appartenant à root** : par défaut, le conteneur écrit ses résultats en tant que root. On lance donc Docker avec `-user $(id -u):$(id -g) -e HOME=/tmp`.
- **Noms de fichiers très longs** : ColabFold nomme les sorties d'après l'en-tête FASTA complet (`sp_P0DP23_CALM1_HUM`). Il faut raccourcir l'en-tête (`>P0DP23`) avant de lancer le calcul.
- **Mémoire** : TensorFlow affiche des avertissements `Allocation exceeds 10% of free system memory`. Ils sont sans conséquence ici : la consommation plafonne à environ 4 Go.
- **Modèle de rapport** : `inputenc[utf8x]` et un `\lstset` faisant référence à des macros non définies empêchaient la compilation ; nous les avons remplacés par `utf8`.

## 4 Test sur une protéine réelle

résultats

## 5 Retour d'expérience

**Ce qui a fonctionné.** L'image Docker de ColabFold a fonctionné du premier coup sur une machine sans GPU. Le serveur MMseqs2 a construit le MSA en quelques secondes. Le point essentiel est que toute la chaîne, de la séquence à la structure puis aux figures, tient en deux commandes (`predict.sh` puis `analyse.py`), sans installation de bases de données.

**Ce qui a posé problème.** Le temps de calcul sur CPU () est le principal inconvénient. Il devient prohibitif au-delà de quelques centaines de résidus, car le coût croît à peu près comme le carré de la longueur. Viennent ensuite des détails pratiques (droits des fichiers, noms trop longs) qui ne sont documentés nulle part de façon centralisée. durée

### Recommandations.

- Sans GPU et pour une protéine de plus de 300 résidus, utiliser le notebook ColabFold sur Google Colab, qui produit exactement les mêmes fichiers ; le script d'analyse s'applique tel quel.
- Pour un premier essai, réduire le calcul avec `-num-models 1`.
- Toujours lire le pLDDT *et* la PAE avant d'interpréter la structure, et ne jamais conclure sur l'orientation relative de deux domaines sans consulter la PAE.
- Le serveur MMseqs2 est une ressource partagée : ne pas soumettre des centaines de séquences d'un coup.

## Utilisation de l'IA générative

Ce projet a été réalisé avec l'aide de Claude (Anthropic), utilisé via Claude Code, pour la comparaison des solutions techniques, la mise en place de l'environnement Docker, l'écriture des scripts `predict.sh` et `analyse.py`, la rédaction du `README.md` et de ce rapport, ainsi que l'interprétation du pLDDT et de la PAE. Tous les contenus ont été relus et vérifiés, en particulier par rapport aux sources citées. Lien public vers la conversation : .

lien de  
partage d  
la conver-  
sation

## Références

- [1] Josh Abramson, Jonas Adler, Jack Dunger, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630 :493–500, 2024.
- [2] Christian B. Anfinsen. Principles that govern the folding of protein chains. *Science*, 181 :223–230, 1973.
- [3] EMBL-EBI Training. AlphaFold : A practical guide. <https://www.ebi.ac.uk/training/online/courses/alphafold/>. Pages « What is the protein folding problem? », « A high-level overview », « pLDDT : Understanding local confidence », « PAE : A measure of global confidence », « Strengths and limitations of AlphaFold 2 ». Consulté le 24/09/2026.
- [4] Google DeepMind. AlphaFold – dépôt officiel de google deepmind. <https://github.com/google-deepmind/alphafold>. Section « Genetic databases » : besoins en disque et en GPU de l’installation complète.
- [5] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 :583–589, 2021. <https://doi.org/10.1038/s41586-021-03819-2>.
- [6] Milot Mirdita, Sergey Ovchinnikov, and Martin Steinegger. ColabFold – dépôt officiel (v1.5.5). <https://github.com/sokrypton/ColabFold>. Image Docker [ghcr.io/sokrypton/colabfold:1.5.5-cuda12.2.2](https://ghcr.io/sokrypton/colabfold:1.5.5-cuda12.2.2); notebook <https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/AlphaFold2.ipynb>.
- [7] Milot Mirdita, Konstantin Schütze, Yoshitaka Moriwaki, Lim Heo, Sergey Ovchinnikov, and Martin Steinegger. ColabFold : making protein folding accessible to all. *Nature Methods*, 19 :679–682, 2022. <https://doi.org/10.1038/s41592-022-01488-1>.
- [8] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35 :1026–1028, 2017.
- [9] Mihaly Varadi, Stephen Anyango, Mandar Deshpande, et al. AlphaFold protein structure database : massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 50(D1) :D439–D444, 2022.